

Cuestionario Tema 2 - Lenguajes para aplicaciones en TR

1. Teniendo en cuenta las definiciones puestas a continuación, seleccione cuál de las siguientes asignaciones en ADA son correctas.

```
type Weight is digits 4 range 0.0 .. 200.0;
```

```
type Height is digits 4 range 0.0 .. 200.0;
```

```
w0, w1: Weight;
```

```
h0, h1: Height;
```

- a. w0 := 75.4;
h0 := 180.0;
h1 := w0 + 100.0; *(No, w0 es distinto tipo que h1)*
- b. w0 := 75.4;
h0 := 180.0;
h1 := w0 + h0/10.0; *(No, w0 es distinto tipo que h1)*
- c. w0 := 75.4;
h0 := 180.0;
h1 := h0 + 52.0; *(No, la suma se sale del rango)*
- d. w0 := 75.4;
h0 := 180.0;
h1 := h0 + h0/10.0;

2. Seleccione qué definiciones y sentencias en ADA se utilizarían para definir un rango de números reales con precisión de milésima entre -99 y 99.

- a. type Dato is digits 5 range -99.999 .. 99.999;
- b. type Dato is digits 0.001 range -99.999 .. 99.999;
- c. type Dato is delta 0.001 range -99.999 .. 99.999;
- d. type Dato is delta 5 range 0.001;

3. Escoja el lenguaje de programación que presenta mayor nivel de simplicidad para obtener un registro concreto de una base de datos.

- a. ADA
- b. C++
- c. SQL
- d. Ensamblador

4. Seleccione el lenguaje de programación que tiene una mayor flexibilidad.

- a. SQL
- b. Python
- c. C
- d. Java

5. Indique qué característica se está potenciando cuando un lenguaje de programación obliga al programador a especificar todos y cada uno de los parámetros de una función.

- a. Modularidad
- b. Legibilidad
- c. Simplicidad
- d. Flexibilidad

6. Seleccione qué característica no favorece la legibilidad.

- a. Poder usar comentarios multilínea.
- b. Poder utilizar mayúsculas y minúsculas en las variables.
- c. Utilizar saltos a direcciones absolutas dentro del código
- d. Poder definir tipos de datos abstractos.

7. El método de Newton es un método matemático iterativo para encontrar los ceros de una función $f(x)$ derivable. Es decir, va aproximando de manera iterativa la resolución de una función para encontrar un valor que haga cero a dicha función. Necesita la función original ($f(x)$) y la primera derivada de la función ($df(x)$). Parte de un valor inicial, habitualmente $x=0$, y obtiene el nuevo valor como $x_{i+1} = x_i + f(x_i)/df(x_i)$. Si la diferencia es inferior a un error determinado, se finaliza.

Tómese *float fabs(float x)* como la función que devuelve el valor absoluto de la variable.

Indique cuál de los siguientes códigos que buscan el 0 de una función por el Método de Newton es preferible en aras de la característica de la eficacia del código.

a. *Correcta*

```
float f (float x); // Function f(x). Externally defined
float df (float x); // Derivative of f(x). Externally defined
float fabs (float x); // Absolute value of x. Externally defined

float x = 0.0, xnew, xold;
float error = 0.001;
int main() {
    xnew = xold = x;
    for (i = 0; i < 1000; i++) {
        xnew = xold + f(xold)/df(xold);
        if (fabs(xnew - xold) < error) break;
    }
    x = xnew;
    printf("%f\n", x);
    return 0;
}
```

b.

```
float f (float x); // Function f(x). Externally defined
float df (float x); // Derivative of f(x). Externally defined
float fabs (float x); // Absolute value of x. Externally defined

float x = 0.0, xnew, xold;
float error = 0.001;
int main() {
    xold = xnew = x;
    iteration: xnew = xold + f(xold)/df(xold);
    if (fabs(xnew - xold) > error)
        goto iteration;

    x = xnew;
    printf("%f\n", x);
    return 0;
}
```

c.

```
float f (float x); // Function f(x). Externally defined
float df (float x); // Derivative of f(x). Externally defined
float fabs (float x); // Absolute value of x. Externally defined

float x = 0.0, xnew, xold;
float error = 0.001;
int main() {
    xold = x;
    xnew = xold + 2*error;
    while (fabs(xnew - xold) < error) {
        xnew = xold + f(xold)/df(xold);
    }

    x = xnew;
    printf("%f\n", x);
    return 0;
}
```

d.

```
float f (float x); // Function f(x). Externally defined
float df (float x); // Derivative of f(x). Externally defined
float fabs (float x); // Absolute value of x. Externally defined

float x = 0.0, xnew, xold;
float error = 0.001;
int main() {
    xold = xnew = x;
    do {
        xnew = xold + f(xold)/df(xold);
    } while (fabs(xnew - xold) > error);

    x = xnew;
    printf("%f\n", x);
    return 0;
}
```

8. De los siguientes lenguajes de programación, seleccione aquel que presente menor nivel de seguridad.

- a. Fortran
- b. C
- c. Python
- d. Ensamblador

9. ¿Qué tipo de sistema sería el más recomendable para poner una aplicación de tiempo real que controle la velocidad de rotación en función del peso en un sistema empotrado dentro de una lavadora inteligente?

- a. Un ejecutable con enlaces dinámicos.
- b. Un ejecutable stand-alone.
- c. Un ejecutable con integración completa de servicios.
- d. Un Sistema Operativo en Tiempo Real con la aplicación como un servicios del Sistema Operativo con carga y descarga a demanda.

10. Indique qué característica del lenguaje hace que sea posible que un mismo programa se pueda ejecutar en un procesador de 32 bits y en uno de 64 bits.

- a. Seguridad
- b. Eficacia
- c. Portabilidad
- d. Flexibilidad

11. Supongamos que se ha producido un error en una función. Si dicha función no tiene manejador de excepciones para la excepción que se ha producido, seleccione cuál de las siguientes acciones se realizaría.

- a. Se comienza a evaluar la pila de llamadas comenzando desde el main o desde el procedimiento donde haya un manejador “catch-all”. En cuanto se encuentre una función con un manejador para la excepción generada, se le da control y en caso de no encontrar, se ejecuta el manejador catch-all o el del Sistema Operativo, si no existía.
- b. Primero, busca si existe un manejador “catch-all” en toda la pila de llamadas de funciones. Si existe, se sube la pila de llamada de funciones hasta ese manejador y se le da control. En caso contrario, se le da control al manejador del Sistema Operativo, que habitualmente aborta la ejecución del proceso.
- c. Primero, busca en la función llamadora de la actual y ejecuta el manejador de excepciones de esa función. Si no existe o no se maneja la excepción que se ha lanzado, se pasa al superior, hasta que llegue al nivel del Sistema Operativo, donde se abortaría la ejecución del proceso.
- d. Primero, evalúa si en toda la pila de llamadas a la función hay algún manejador activo. Si hay uno, se le da control a él. Si hay más de uno, se ejecuta el que esté más arriba de la pila, cuando finaliza, se ejecuta el siguiente, hasta que todos los manejadores se terminan de ejecutar. Si no hay ninguno, se ejecuta el manejador del Sistema Operativo, abortando la ejecución del proceso.

12. En ADA, ¿qué característica se favorece al permitir indicar un número de la siguientes manera 2#1001_0001#?

- a. Portabilidad
- b. Modularidad
- c. Simplicidad
- d. Legibilidad

13. ¿Qué debe proporcionar un lenguaje de programación en relación a los errores que se produzcan mientras los programas se ejecutan?

- a. Un manejador “catch-all” que capture todos los errores en el nivel del main.
- b. Un soporte de gestión de excepciones flexible, con baja sobrecarga y de fácil uso.
- c. Un mecanismo de fail-safe que permita cerrar la aplicación de manera segura.
- d. Compiladores que permitan detectar posibles errores en tiempo de compilación y, en su caso, permitan identificar funciones que no tengan manejadores de excepciones que puedan lanzarse en un determinado trozo de código.

14. Indique cuál de las siguientes sentencias daría error al utilizar un lenguaje con tipificación fuerte de datos.

- a. *No, porque ‘max’ podría hacer los castings correctos internamente.*

```
float max (int v0, char v1);

int main() {
    int a = 1;
    char b = 2;

    float c;

    c = max(a, b);

    return 0;
}
```

- b. *No, porque ‘transf’ podría hacer los castings internamente.*

```
float transf(int v0);
float transf(char v1);

int main() {
    int a = 1, b = 2;

    float c;

    c = transf(a) + transf(b);

    printf("%f\n", c);

    return 0;
}
```

- c. *Sí, porque compara datos de distinto tipo sin hacer casting explícito.*

```
int equal (int a, char b) {
    if (a == b)
        return 1;

    else
        return 0;
}
```

d. *No, porque se hacen los castings correctos.*

```
int main() {  
    int a = 1;  
    char b = 1;  
  
    float c;  
  
    c = (float) (a + (int)b);  
    if (c == 2.0)  
        printf("Ok!");  
    else  
        printf("Error!");  
    return 0;  
}
```

15. ¿Qué característica es la que se busca en un lenguaje de programación que sea capaz de proporcionar una medida precisa de la computación de los diferentes módulos?

- a. **Eficacia**
- b. Simplicidad
- c. Modularidad
- d. Portabilidad

16. Indica cuál de los siguientes códigos de datos podría tener problemas de portabilidad dependiendo del sistema en el que se ejecute.

a. *No, porque independientemente de si el casting de 'int' trunca o redondea, el valor de 'b' siempre va a ser 2. Nunca dará error.*

```
#include <stdio.h>  
  
int main() {  
    float a = 1.9;  
    int b;  
  
    b = (int) (a + 0.5);  
  
    if (b == 2)  
        printf("Ok!");  
    else  
        printf("Error!");  
    return 0;  
}
```

b. *Podría dar error, pero no depende sólo del sistema.*

```
#include <stdio.h>

int main() {
    float a = 1.9;
    int b;

    b = (int)(a) + 1;

    if (b == 2)
        printf("Ok!");
    else
        printf("Error!");
    return 0;
}
```

c. *El resultado de 'b' depende exclusivamente de cómo se realice el casting de 'int'. Por tanto, el sistema donde se ejecute es un factor muy importante.*

```
#include <stdio.h>

int main() {
    float a = 1.9;
    int b;

    b = (int)(a);

    if (b == 1)
        printf("Ok!");
    else
        printf("Error!");
    return 0;
}
```

d. *No, porque al no ser necesario un casting, el resultado será el mismo independientemente del sistema en el que se ejecute. Nunca dará error.*

```
#include <stdio.h>

int main() {
    float a = 1.9, b;

    b = a + 0.1;

    if (b == 2.0)
        printf("Ok!");
    else
        printf("Error!");
    return 0;
}
```